

Laborator 03 –Extins Clase simple în Java

Programare Orientată pe Obiecte

October 10, 2025

Obiective

Pentru acest laborator extins obiectivele de învățare sunt următoarele:

- să învățați să descrieți structura generală a unei **clase** în Java;
- să **instanțiați obiecte** folosind mai mulți **constructori**;
- să utilizați **metode getter și setter**;
- să scrieți metoda **equals** pentru compararea obiectelor;
- să proiectați clase simple pentru forme geometrice (fără moștenire).

1 Concepte de bază

Clasă O clasă este un *tip* definit de programator care descrie **starea** (câmpuri/atribute) și **comportamentul** (metode) unor obiecte.

Obiect Un obiect este o *instanță* a unei clase, creată în memorie prin **new**. Fiecare obiect are propriile valori pentru atribute.

Constructor O metodă specială, cu același nume ca al clasei, care inițializează un obiect nou. Într-o clasă poate exista *mai mult de un* constructor (supraincărcare).

Notă: În exemplele de mai jos evităm deocamdată cuvântul cheie **this**. Pentru a nu fi nevoie de **this**, vom folosi nume de parametri *diferite* de numele câmpurilor.

Metodă O funcție definită într-o clasă. Metodele **getter** expun valori ale câmpurilor; metodele **setter** modifică valori ale câmpurilor (de regulă după validări simple).

2 Structura generală a unei clase

Exemplu pentru structura unei clase este dat mai jos:

Listing 1: Schemă pentru o clasă Java

```
1 public class Punct {  
2     private int x;
```

```

3     private int y;
4
5     // Constructor fara parametri
6     public Punct() {
7         x = 0;
8         y = 0;
9     }
10
11    // Constructor cu parametri
12    public Punct(int coordX, int coordY) {
13        x = coordX;
14        y = coordY;
15    }
16
17    // Metode getter
18    public int getX() {
19        return x;
20    }
21
22    public int getY() {
23        return y;
24    }
25
26    // Metode setter
27    public void setX(int nouX) {
28        x = nouX;
29    }
30
31    public void setY(int nouY) {
32        y = nouY;
33    }
34
35    // Suprascriere equals pentru comparare logica
36    @Override
37    public boolean equals(Object o) {
38        if (o == null) return false;
39        if (getClass() != o.getClass()) return false;
40        Punct altPunct = (Punct) o;
41        return x == altPunct.x && y == altPunct.y;
42    }
43 }

```

Instanțierea obiectelor Exemplu de instanțiere de obiecte:

Listing 2: Crearea și utilizarea obiectelor

```

1 public class MainPunct {
2     public static void main(String[] args) {
3         // Crearea unor obiecte de tip Punct
4         Punct p1 = new Punct(); // (0, 0)
5         Punct p2 = new Punct(2, 3); // (2, 3)
6         Punct p3 = new Punct(2, 3); // (2, 3)
7         Punct p4 = new Punct(5, 1); // (5, 1)
8
9         // Afisarea coordonatelor
10        System.out.println("P1: (" + p1.getX() + ", " + p1.getY() + ")");
11        System.out.println("P2: (" + p2.getX() + ", " + p2.getY() + ")");
12        System.out.println("P3: (" + p3.getX() + ", " + p3.getY() + ")");

```

```

13         System.out.println("P4: (" + p4.getX() + ", " + p4.getY() + ")");
14
15         // Compararea punctelor
16         System.out.println("p1.equals(p2)? " + p1.equals(p2)); // false
17         System.out.println("p2.equals(p3)? " + p2.equals(p3)); // true
18         System.out.println("p3.equals(p4)? " + p3.equals(p4)); // false
19
20         // Modificarea unui punct
21         p1.setX(2);
22         p1.setY(3);
23         System.out.println("Dupa modificare, p1.equals(p2)? " + p1.equals(p2)); // true
24     }

```

3 Cerințe de laborator

3.1 Problema 1:

Fie clasa Dreptunghi definită mai jos.

```

1  public class Dreptunghi {
2      private double latime;
3      private double inaltime;
4
5      public Dreptunghi() {
6          latime = 0.0;
7          inaltime = 0.0;
8      }
9
10     public Dreptunghi(double lat, double inalt) {
11         latime = lat;
12         inaltime = inalt;
13     }
14
15     public double getLatime() { return latime; }
16     public double getInaltime() { return inaltime; }
17
18     public void setLatime(double lat) { latime = lat; }
19     public void setInaltime(double inalt) { inaltime = inalt; }
20
21     @Override
22     public boolean equals(Object o) {
23         if (o == null) return false;
24         if (getClass() != o.getClass()) return false;
25         Dreptunghi d = (Dreptunghi) o;
26         return latime == d.latime && inaltime == d.inaltime;
27     }
28 }

```

Scrieti codul pentru o clasa MainClass in care in metoda main sa instantiati mai multe dreptunghiuri si apoi sa verificati daca două dreptunghiuri sunt egale.

3.2 Problema 2:

Definiți clasele Cerc (cu attribute centru de tip *Punct* si raza) si clasa Patrat (cu atributul latura).

1. Creati mai multi constructori (cel puțin doi) pentru fiecare din clasele Cerc si Patrat.

2. În clasa `MainClass` instanțiați mai multe `Cercuri` și `Patrate`.
3. Faceți metode `getter` și `setter` pentru toate campurile claselor
4. Scrieti metode de calculare a ariei și a perimetrului pentru fiecare din cele trei clase.
5. Verificați egalitatea a două cercuri și a două Pătrate ca și în clasa `Dreptunghi`.
6. Comparați două cercuri pe baza ariei.

3.3 Problema 3:

Actualizați clasele `Cerc`, `Patrat` și `Dreptunghi` și adăugați la fiecare următoarele atribute: **culoare** (`String`) și **poziție** (un obiect de tip `Punct` cu coordonate întregi).

1. Adăugați constructori în cele trei clase ca să inițializați și culoarea și poziția.
2. În clasa `MainClass` instanțiați mai multe `Cercuri`, `Patrate` și `Dreptunghiuri`.
3. Faceți metode `getter` și `setter` pentru toate campurile adăugate.
4. Actualizați metoda `equals()` din fiecare clasă astfel încât să considerați și poziția și culoarea când comparați două obiecte.
5. în main comparați diferite obiecte ca să testați funcționalitatea metodei `equals()` actualizată.